

ИНСТИТУТ ЛАЗЕРНЫХ И ПЛАЗМЕННЫХ ТЕХНОЛОГИЙ
Кафедра прикладной математики №31

ОТЧЕТ
по научно-исследовательской работе за 6 семестр
на тему:
Автоматизация аналитического исследования нелинейных
дифференциальных уравнений

Группа:
Студент:
Научный руководитель:
старший преподаватель каф. №31

Б23-215
А.К. Кораблёва
А.А. Кутуков

(подпись)

Аннотация

Целью работы является создание набора алгоритмических инструментов для преодоления трудоемкости ручных символьных выкладок при исследовании нелинейных дифференциальных уравнений. Разрабатываемый программный комплекс включает два независимых модуля:

- универсальный анализатор сингулярностей, выполняющий автоматическое построение многоугольников Ньютона для анализа носителя уравнения и геометрической оценки главного члена асимптотического разложения (порядка полюса);
- специализированный решатель уравнения Курамото–Сивашинского, реализующий алгоритмический поиск точных аналитических решений методами анзацев Риккати и эллиптических функций Вейерштрасса.

Совместное или автономное использование данных модулей позволяет автоматизировать рутинные этапы исследования, минимизировать ошибки вычислений и существенно расширить возможности аналитического моделирования в задачах теории волн и математической физики.

Содержание

1.	Введение	4
2.	Уравнение Курамото-Сивашинского	6
2.1.	Математическая модель и редукция к ОДУ	6
2.2.	Решение аналитическими методами: анзацы Риккати и Вейерштрасса	7
2.2.1	Определение порядка полюса уравнения Курамото-Сивашинского	7
2.2.2.	Аналитическое решение методом Риккати	8
2.2.3.	Аналитическое решение методом Вейерштрасса	9
2.3.	Алгоритмизация символьных вычислений и переход к Python/SymPy	11
2.4	Визуализация и качественный анализ волновых профилей	12
2.4.1	Построение графиков решений	12
2.4.2	Анализ переходов между режимами	15
2.6.	Верификация результатов и обсуждение вычислительной эффективности	15
3.	Геометрический метод многоугольников Ньютона для определения порядка полюса решений нелинейных дифференциальных уравнений	17
3.1.	Математические основы метода: носитель дифференциального многочлена	17
3.2.	Определение порядка полюса через наклон доминирующей грани	18
3.3.	Алгоритм и программная реализация модуля сингулярного анализа	19
3.4.	Обратная задача: генерация тестовых уравнений по заданному носителю	20
3.6.	Верификация алгоритма и вычислительный эксперимент	21
3.7.	Выводы	24
4.	Заключение	26
5.	Список литературы и интернет-ресурсов	28
	Приложение А. Доступ к исходным материалам	29

1. Введение

Нелинейные дифференциальные уравнения (НДУ) составляют математический фундамент описания сложных физических, биологических и инженерных процессов: от турбулентности и горения до нелинейной оптики и динамики популяций. В отличие от линейных моделей, аналитические решения НДУ встречаются редко, однако их наличие критически важно для понимания структуры фазового пространства, анализа устойчивости, верификации численных схем и конструирования управляемых волновых структур. Традиционные методы поиска точных решений — анализ Пенлеве, метод доминантного баланса, анзацы Риккати и эллиптической функции Вейерштрасса — обладают высокой эффективностью, но их ручная реализация сопряжена с громоздкими символьными выкладками, высокой вероятностью вычислительных ошибок и требует глубокой математической подготовки.

В современных условиях наблюдается острая потребность в автоматизации символьных вычислений. Коммерческие компьютерные алгебраические системы, такие как Maple, традиционно используются для прототипирования и проверки корректности аналитических алгоритмов. Однако их коммерческая модель распространения и ограничения на интеграцию с открытыми научными инструментами обуславливают необходимость переноса алгоритмов в свободную экосистему Python. Результатом данной работы стал набор программных модулей, объединяющих поиск точных решений уравнения Курамoto–Сивашинского и геометрический сингулярный анализ на базе многоугольников Ньютона.

Целью настоящей научно-исследовательской работы является алгоритмизация и программная реализация методов аналитического исследования нелинейных дифференциальных уравнений на базе открытых технологий. Для достижения поставленной цели решаются следующие задачи:

1. Изучить теоретические основы метода многоугольников Ньютона и его применение для определения порядка полюсов решений нелинейных дифференциальных уравнений.
2. Разработать теоретический аппарат аналитического решения уравнения Курамoto–Сивашинского и реализовать поиск точных решений методами анзацев Риккати и эллиптической функции Вейерштрасса (с первоначальной верификацией в среде Maple).
3. Перенести верифицированные алгоритмы в среду Python с использованием библиотеки SymPy, создав специализированный программный модуль для поиска точных решений уравнения Курамoto–Сивашинского.
4. Разработать универсальный программный модуль для автоматического построения носителя дифференциального многочлена, формирования многоугольника Ньютона и геометрического вычисления порядка главного члена асимптотического разложения (порядка полюса).
5. Сформировать верификационный набор тестов для модуля многоугольников Ньютона и провести отладку алгоритма на классических уравнениях математической физики (уравнения Пенлеве, Кортевега–де Фриза, Бюргерса и др.).

Научная новизна работы заключается в алгоритмизации метода многоугольников Ньютона для сингулярного анализа нелинейных дифференциальных уравнений в открытой среде Python. Разработанный алгоритм корректно обрабатывает носители дифференциальных многочленов, строит их верхнюю выпуклую оболочку и на

основе наклона доминирующих граней аналитически определяет порядок полюса, что обеспечивает надежную и формализованную альтернативу ручным выкладкам классического метода доминантного баланса.

Практическая значимость работы обусловлена возможностью использования разработанных инструментов для быстрой априорной оценки порядка полюсов новых нелинейных моделей перед применением трудоемких методов точного интегрирования. Автоматизация рутинных символьных операций позволяет исследователям сосредоточиться на физической интерпретации результатов.

Структура отчёта выстроена в соответствии с логикой разработки двух независимых программных модулей. В первой главе рассматривается уравнение Курамото–Сивашинского: приводятся аналитические выкладки для методов Риккати и Вейерштрасса, демонстрируется поиск и визуализация точных решений, а также представлена полная реализация на Python/SymPy с интерактивным интерфейсом для работы с полученными семействами решений. Во второй главе излагаются теоретические основы метода многоугольников Ньютона, описывается алгоритм формирования носителя уравнения и построения выпуклой оболочки для определения порядка полюса. Приводится программная реализация данного подхода на Python и результаты его автоматизированного тестирования на наборе классических уравнений математической физики. Заключительная часть содержит выводы по выполненной работе и перспективы дальнейшего развития предложенного вычислительного комплекса.

2. Уравнение Курамото-Сивашинского

2.1. Математическая модель и редукция к ОДУ

В основе предлагаемого алгоритмического подхода лежит метод простейших уравнений, подробно изложенный в монографии Н. А. Кудряшова «Методы нелинейной математической физики». Данный метод представляет собой универсальный конструктивный аппарат для поиска точных решений нелинейных дифференциальных уравнений. Его основная идея заключается в замене исходной неизвестной функции на решение некоторого вспомогательного дифференциального уравнения более низкого порядка, которое допускает аналитическое интегрирование (так называемого простейшего уравнения). В качестве простейших уравнений чаще всего выступают уравнение Риккати $Y' = b - Y^2$, уравнение для эллиптической функции Вейерштрасса $(R')^2 = -4R^3 + aR^2 + 2bR + c$, логистическое уравнение или линейные уравнения с постоянными коэффициентами. Подстановка такого анзаца преобразует исходное нелинейное ОДУ в полиномиальную систему алгебраических уравнений относительно неизвестных коэффициентов разложения и параметров уравнения.

Объектом исследования в данной работе выступает уравнение Курамото-Сивашинского:

$$u_t + uu_x + u_{xx} + \sigma u_{xxx} + u_{xxxx} = 0, \quad (1)$$

которое является одной из фундаментальных моделей нелинейной математической физики. Уравнение описывает динамику фронтов пламени, течение тонких плёнок жидкости на наклонных плоскостях, а также процессы в реакционно-диффузионных системах и плазме. Физико-математическая структура уравнения (1) отражает конкуренцию нескольких фундаментальных механизмов:

- Нелинейность (uu_x) отвечает за конвективный перенос и формирование крутых фронтов (эффект обострения);
- Диссипация/Антидиссипация (u_{xx}) в данном контексте играет дестабилизирующую роль, приводя к длинноволновой неустойчивости;
- Дисперсия (σu_{xxx}) отвечает за зависимость фазовой скорости волны от её длины, способствуя осцилляциям профиля;
- Высшая диссипация (u_{xxxx}) обеспечивает стабилизацию системы на малых масштабах, предотвращая коллапс решений и ограничивая спектр неустойчивых мод.

Взаимодействие этих эффектов порождает богатую динамику, включая хаотические режимы и образование устойчивых когерентных волновых структур.

Для поиска точных решений уравнение (1) редуцируется к обыкновенному дифференциальному уравнению с помощью стандартного анзаца бегущей волны:

$$u(x, t) = y(z), \quad z = x - C_0 t, \quad (2)$$

где C_0 — скорость распространения волны. Подставляя производные

$$u_t = -C_0 y', \quad u_x = y', \quad u_{xx} = y'', \quad u_{xxx} = y''', \quad u_{xxxx} = y'''' ,$$

в уравнение (1), получаем нелинейное ОДУ четвёртого порядка:

$$y'''' + \sigma y''' + y'' + yy' - C_0 y' = 0. \quad (3)$$

Интегрируя это уравнение один раз по переменной z и вводя постоянную интегрирования C_1 , приходим к автономному ОДУ третьего порядка:

$$y''' + \sigma y'' + y' + \frac{1}{2}y^2 - C_0 y + C_1 = 0. \quad (4)$$

Именно уравнение (4) является основным объектом дальнейшей аналитической и компьютерной обработки.

Параметры уравнения (4) играют ключевую роль в формировании волновых профилей и определяют условия существования точных решений:

- Параметр C_0 определяет скорость бегущей волны в лабораторной системе координат. Его значение напрямую влияет на баланс между нелинейным членом $\frac{1}{2}y^2$ и линейным членом $-C_0 y$, определяя амплитуду и асимптотическое поведение решения при $z \rightarrow \pm\infty$.
- Параметр σ характеризует интенсивность дисперсионных эффектов. При $\sigma = 0$ уравнение переходит в классическую форму Курамото–Сивашинского. Отличие σ от нуля модифицирует фазовую структуру решений, приводя к появлению асимметричных профилей и изменению условий существования солитонов.
- Параметр C_1 является константой интегрирования, возникающей при редукции к ОДУ. Он задаёт фоновое значение решения (средний уровень волны) и сдвигает потенциальную яму в соответствующей динамической системе, влияя на тип сепаратрис и, следовательно, на качественную форму получаемых точных решений (солитоны, кноидальные волны, рациональные решения).

Совокупность этих параметров определяет область существования точных решений, что делает задачу их аналитического нахождения чувствительной к выбору начальных алгебраических условий, решаемую в рамках автоматизированной реализации метода простейших уравнений.

2.2. Решение аналитическими методами: анзацы Риккати и Вейерштрасса

После редукции уравнения Курамото–Сивашинского к автономному ОДУ третьего порядка

$$y''' + \sigma y'' + y' + \frac{1}{2}y^2 - C_0 y + C_1 = 0, \quad (5)$$

возникает задача построения его точных решений. Поскольку уравнение содержит полиномиальную нелинейность (в данном случае квадратичную y^2), эффективным инструментом является метод простейших уравнений. Степень полинома в анзаце определяется не степенью нелинейности, а порядком подвижного полюса решения p , что гарантирует конечность разложения и позволяет свести задачу к решению системы алгебраических уравнений.

2.2.1 Определение порядка полюса уравнения Курамото–Сивашинского

Порядок полюса p находится методом доминантного баланса. В окрестности подвижной особенности полагаем асимптотику $y(z) \sim A\xi^{-p}$, где $\xi = z - z_0 \rightarrow 0$. Доминирующими слагаемыми в (5) являются старшая производная y''' и нелинейный

член $\frac{1}{2}y^2$. Их ведущие показатели:

$$y''' \sim \xi^{-p-3}, \quad \frac{1}{2}y^2 \sim \xi^{-2p}. \quad (6)$$

Условие баланса $\xi^{-p-3} \sim \xi^{-2p}$ даёт алгебраическое уравнение

$$-p - 3 = -2p \Rightarrow p = 3. \quad (7)$$

Поскольку $p \in \mathbb{Z}_{>0}$, особенность является полюсом третьего порядка. Это обосновывает использование полиномиальных анзацев третьей степени по решениям простейших уравнений.

2.2.2. Аналитическое решение методом Риккати

В качестве простейшего уравнения выбираем уравнение Риккати:

$$Y' = b - Y^2, \quad b = \text{const}. \quad (8)$$

Его общее решение при $b > 0$ имеет вид $Y(z) = \sqrt{b} \tanh(\sqrt{b}(z + z_0))$. С учётом $p = 3$ ищем решение уравнения (4) в виде кубического полинома:

$$y(z) = A_0 + A_1Y + A_2Y^2 + A_3Y^3. \quad (9)$$

Вычисляем производные y , последовательно используя правило замены $d/dz = (b - Y^2)d/dY$:

$$y' = -3A_3Y^4 - 2A_2Y^3 + (3A_3b - A_1)Y^2 + 2A_2bY + A_1b, \quad (10)$$

$$y'' = 12A_3Y^5 + 10A_2Y^4 - 12A_3bY^3 - 6A_2bY^2 + (3A_3b^2 - 3A_1b)Y + 2A_2b^2, \quad (11)$$

$$y''' = -60A_3Y^6 - 50A_2Y^5 + (60A_3b - 6A_1)Y^4 + 20A_2bY^3 + \dots \quad (12)$$

Подстановка y, y', y'', y''' в уравнение (4) и раскрытие скобок приводит к полиному относительно Y . Приравнявая коэффициенты при каждой степени Y^k ($k = 0, \dots, 6$) к нулю, получаем переопределённую систему алгебраических уравнений относительно неизвестных $A_0, A_1, A_2, A_3, b, \sigma, C_0, C_1$.

Решение системы показывает, что $A_3 = 120$, $A_2 = -15\sigma$, а параметры b и σ связаны дискретными соотношениями. В зависимости от значения σ получаем следующие семейства точных решений:

1. Случай $\sigma = \pm 4$, $b = 1/4$:

$$y(z) = C_0 \mp 9 - 30Y + 60(\pm 1)Y^2 + 120Y^3, \quad Y(z) = \frac{1}{2} \tanh\left(\frac{z + z_0}{2}\right). \quad (13)$$

Данные решения описывают уединённые волны (солитоны) с гладким переходом между асимптотиками $y(\pm\infty) = C_0 \mp 9$.

2. Случай $\sigma = 0$, $b = 11/76$:

$$y(z) = C_0 - \frac{270}{19}Y + 120Y^3, \quad Y(z) = \sqrt{\frac{11}{76}} \tanh\left(\sqrt{\frac{11}{76}}(z + z_0)\right). \quad (14)$$

3. Случай $\sigma = \pm \frac{12\sqrt{47}}{47}$, $b = 1/188$:

$$y(z) = C_0 \mp \frac{45\sqrt{47}}{2209} + \frac{90}{47}Y \pm \frac{180\sqrt{47}}{47}Y^2 + 120Y^3, \quad Y(z) = \frac{1}{2\sqrt{47}} \tanh\left(\frac{z+z_0}{2\sqrt{47}}\right). \quad (15)$$

4. Случай $\sigma = \pm \frac{16\sqrt{73}}{73}$, $b = 1/292$:

$$y(z) = C_0 \mp \frac{60\sqrt{73}}{5329} + \frac{150}{73}Y \pm \frac{240\sqrt{73}}{73}Y^2 + 120Y^3, \quad Y(z) = \frac{1}{2\sqrt{73}} \tanh\left(\frac{z+z_0}{2\sqrt{73}}\right). \quad (16)$$

2.2.3. Аналитическое решение методом Вейерштрасса

Для построения более общих эллиптических решений используем анзац, содержащий как саму функцию $R(z)$, так и её производную $R'(z)$:

$$y(z) = A_0 + A_1R(z) + A_2R'(z). \quad (17)$$

В качестве определяющего уравнения для $R(z)$ выбираем уравнение первого рода для эллиптической функции Вейерштрасса:

$$(R')^2 = -4R^3 + aR^2 + 2bR + c. \quad (18)$$

Дифференцируя соотношение (18) по z , выражаем старшие производные R через базисные функции R и R' :

$$\begin{aligned} R'' &= -6R^2 + aR + b, \\ R''' &= -12RR' + aR', \\ R^{(4)} &= -12(R')^2 - 12RR'' + aR''. \end{aligned}$$

Вычислим производные функции $y(z)$ из анзаца (17):

$$\begin{aligned} y' &= A_1R' + A_2R'', \\ y'' &= A_1R'' + A_2R''', \\ y''' &= A_1R''' + A_2R^{(4)}. \end{aligned}$$

Подставляя выражения для $R'', R''', R^{(4)}$ и используя уравнение (18) для исключения $(R')^2$, все производные y', y'', y''' могут быть представлены в виде полиномов от переменных R и R' .

Подставим y, y', y'', y''' в уравнение Курамото–Сивашинского (4). После раскрытия скобок и приведения подобных слагаемых, левая часть уравнения становится полиномом по базису $\{1, R, R^2, R^3, R', RR', (R')^2\}$. Используя (18) для замены $(R')^2$ на полином третьей степени от R , приводим выражение к полиному от R и R' .

Условие обращения этого полинома в тождественный ноль эквивалентно системе алгебраических уравнений на коэффициенты A_i и параметры уравнения. Приравняв коэффициенты при старших степенях R (в частности, при R^3 и R^2) к нулю, определяем ведущие коэффициенты анзаца:

$$A_2 = 60, \quad A_1 = 15\sigma. \quad (19)$$

Далее, приравнивая коэффициенты при R' , RR' , R^2 , R и свободный член, находим связь между параметрами уравнения Вейерштрасса и параметрами уравнения Курамото–Сивашинского:

$$\sigma = \pm 4, \quad b = \frac{1}{24} - \frac{a^2}{24}. \quad (20)$$

Параметр C определяется из условия баланса при R^0 :

$$c = \frac{C_0^2}{2160} - \frac{C_1}{1080} + \frac{a^3}{432} - \frac{a}{144} - \frac{13}{1080}. \quad (21)$$

Таким образом, получаем, что уравнение Курамото–Сивашинского при $\sigma = \pm 4$ имеет решение в виде

$$y(z) = C_0 \mp 5a \mp 1 \pm 60R + 60R_z, \quad (22)$$

где $R(z)$ является решением уравнения (18) с подставленными найденными параметрами.

Для явного вида решения необходимо выразить функцию $R(z)$ через эллиптические функции Якоби. Рассмотрим кубический многочлен в правой части (18):

$$-4R^3 + aR^2 + 2bR + c = -4(R - R_1)(R - R_2)(R - R_3) = 0, \quad (23)$$

где R_1, R_2, R_3 — действительные корни, упорядоченные как $R_1 \geq R_2 \geq R_3$.

В этом случае периодическое решение уравнения (18) записывается через квадрат эллиптического косинуса cn :

$$R(z) = R_2 + (R_1 - R_2) \text{cn}^2 \left(\sqrt{R_1 - R_3} z, S \right), \quad (24)$$

где S — модуль эллиптической функции, определяемый соотношением:

$$S^2 = \frac{R_1 - R_2}{R_1 - R_3}. \quad (25)$$

Для подстановки в анзац (17) требуется также производная $R'(z)$. Дифференцируя выражение для $R(z)$, получаем:

$$R'(z) = -2(R_1 - R_2) \sqrt{R_1 - R_3} \text{cn}(\chi, S) \text{sn}(\chi, S) \text{dn}(\chi, S), \quad (26)$$

где $\chi = \sqrt{R_1 - R_3} z$, а sn и dn — эллиптические синус и дельта-амплитуда соответственно.

Таким образом, общее решение уравнения Курамото–Сивашинского методом Вейерштрасса принимает вид:

$$y(z) = A_0 + A_1 \left[R_2 + (R_1 - R_2) \text{cn}^2(\chi, S) \right] - 2A_2(R_1 - R_2) \sqrt{R_1 - R_3} \text{cn} \text{sn} \text{dn}. \quad (27)$$

Параметр модуля S определяет форму волны:

- При $S \rightarrow 0$ (когда $R_1 \rightarrow R_2$) эллиптический косинус переходит в обычный тригонометрический косинус: $\text{cn}(u, 0) = \cos u$. Решение описывает гармонические колебания.
- При $S \rightarrow 1$ (когда $R_2 \rightarrow R_3$) функция $\text{cn}(u, 1)$ вырождается в гиперболический секанс: $\text{cn}(u, 1) = \text{sech } u = 1/\cosh u$. В этом пределе период волны стремится к бесконечности, и мы получаем уединённую волну (солитон).

2.3. Алгоритмизация символьных вычислений и переход к Python/SymPy

На начальном этапе исследования верификация гипотез и поиск коэффициентов выполнялись в среде Maple. Процесс включал ручную подстановку анзаца в дифференциальное уравнение, раскрытие скобок и приведение подобных слагаемых. Основная трудность заключалась в автоматизации сбора коэффициентов при степенях вспомогательной функции $Y(z)$ или $R(z)$, что требовало значительных временных затрат и ручного контроля при изменении структуры уравнения или порядка анзаца.

Для создания универсального и воспроизводимого инструмента алгоритм был перенесен в среду Python с использованием библиотеки символьной математики SymPy. Архитектура разработанного решателя (функция `solve_riccati_symbolic`) построена на принципах модульности и разделяет процесс вычисления на несколько независимых функциональных блоков:

1. Формирование анзаца и производных. Функция `construct_ansatz` генерирует полиномиальное выражение для $y(z)$ заданной степени p . Для корректного вычисления производных используется функция `compute_derivatives`, которая применяет правило цепной дифференциации с учетом определяющего уравнения Риккати ($Y' = b - Y^2$). Производные высших порядков вычисляются рекурсивно через замену $d/dz = (b - Y^2)d/dY$, что позволяет избежать громоздких выражений в терминах z и работать исключительно в базисе Y .
2. Подстановка и приведение уравнения. Основная процедура `substitute_ode` подставляет полученный анзац и его производные в уравнение Курамото–Сива-шинского. После подстановки применяется метод `sp.expand` для полного раскрытия скобок и приведения выражения к полиномиальному виду относительно вспомогательной переменной Y .
3. Извлечение алгебраической системы. Функция `extract_coefficients` анализирует полученный полином. Используя метод `sp.coeff`, она последовательно извлекает коэффициенты при каждой степени Y^k (от $k = 0$ до $k = 2p$) и приравняет их к нулю. Это формирует переопределенную систему алгебраических уравнений относительно неизвестных параметров A_i , b , σ и констант интегрирования C_0, C_1 .
4. Решение и фильтрация результатов. Система передается в функцию `sp.solve`. Поскольку система переопределена, решатель автоматически проверяет условия совместимости. В результате нахождения решений формируются зависимости между параметрами (например, дискретные значения $\sigma = \pm 4$). Для повышения эффективности реализован блок постобработки `filter_solutions`, который:
 - Отсеивает тривиальные решения (например, $y = 0$).
 - Объединяет дубликаты, возникающие при различных путях символьных преобразований.
 - Группирует семейства решений по типу нелинейности (солитоны, периодические волны).

Важным аспектом реализации стала интеграция с библиотекой визуализации `matplotlib`. Функция `plot_solution` принимает найденные символьные коэффициенты и строит графики волновых профилей, позволяя пользователю интерактивно

исследовать влияние параметров σ и b на форму волны. Интерфейс взаимодействия с пользователем реализован в виде консольного меню, обеспечивающего пошаговый доступ ко всем этапам: от выбора типа анзаца до построения графика найденного решения.

2.4 Визуализация и качественный анализ волновых профилей

В данном разделе проводится численная визуализация найденных аналитических решений уравнения Курамoto–Сивашинского и качественный анализ их поведения при варьировании управляющих параметров.

2.4.1 Построение графиков решений

Для визуализации решений, полученных методами Риккати и Вейерштрасса, использовалась библиотека `matplotlib` в среде Python. Все графики строятся для переменной бегущей волны $z = x - C_0 t$, что позволяет анализировать стационарные профили в движущейся системе координат.

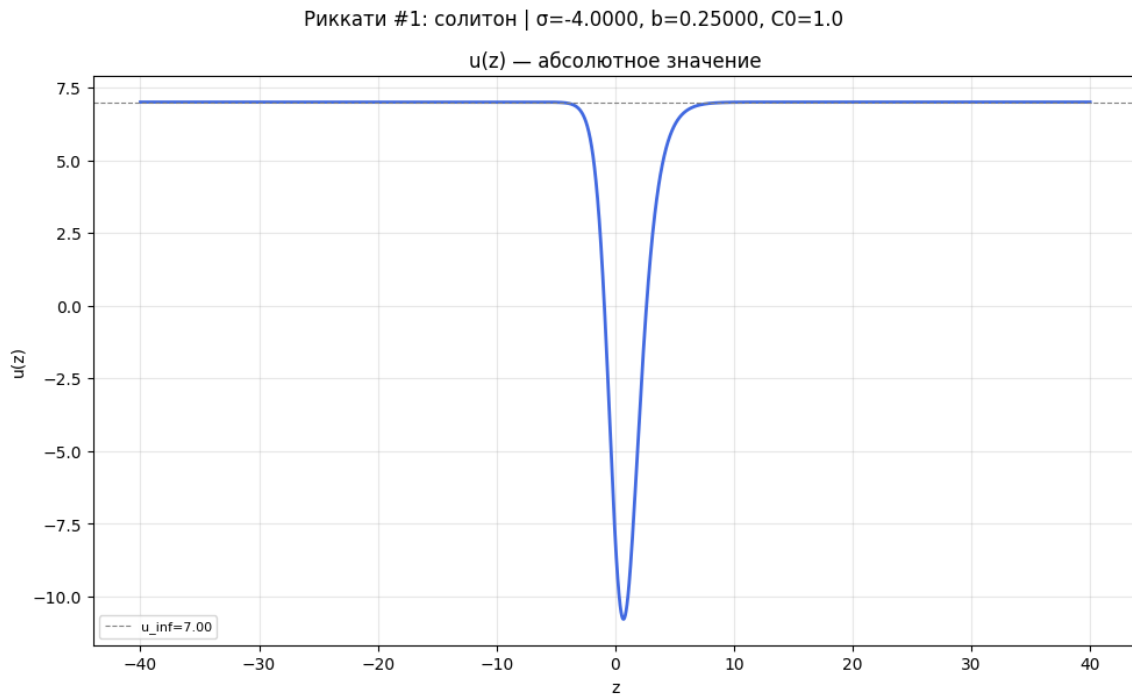


Рис. 1. Локализованный солитонный профиль решения Риккати при $\sigma = -4$, $b = 1/4$, $C_0 = 1$. Видна характерная асимптотика $y(z) \rightarrow u_\infty$ при $|z| \rightarrow \infty$.

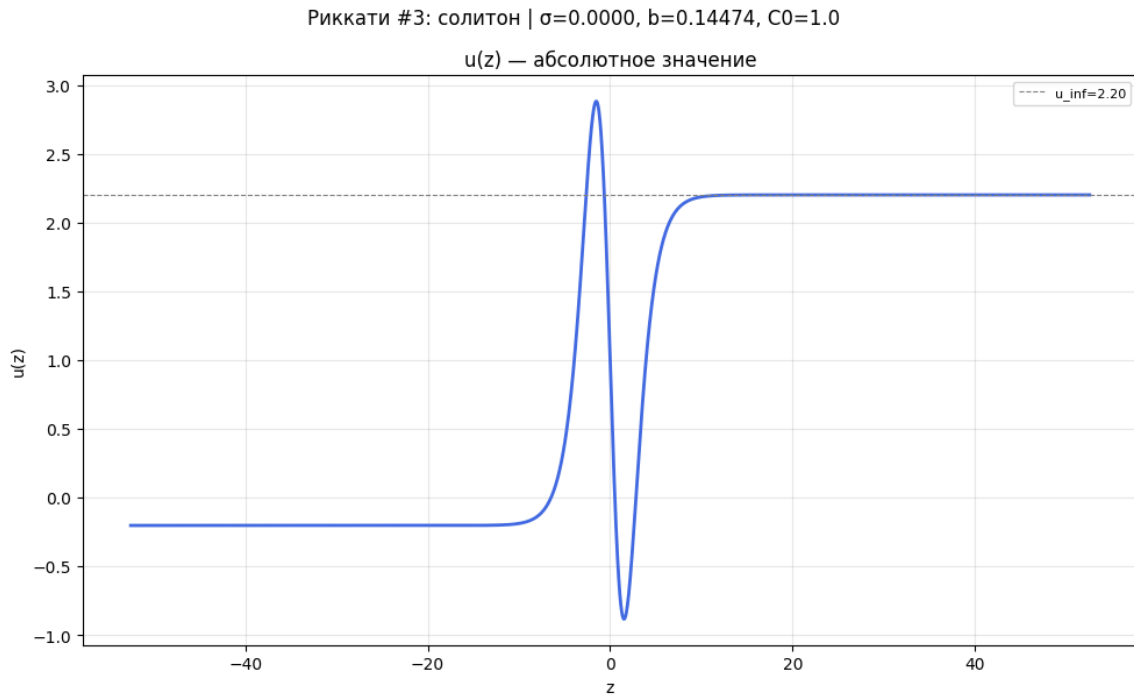


Рис. 2. Локализованный солитонный профиль решения Риккати при $\sigma = 0$, $b \approx 0.145$, $C_0 = 1$.

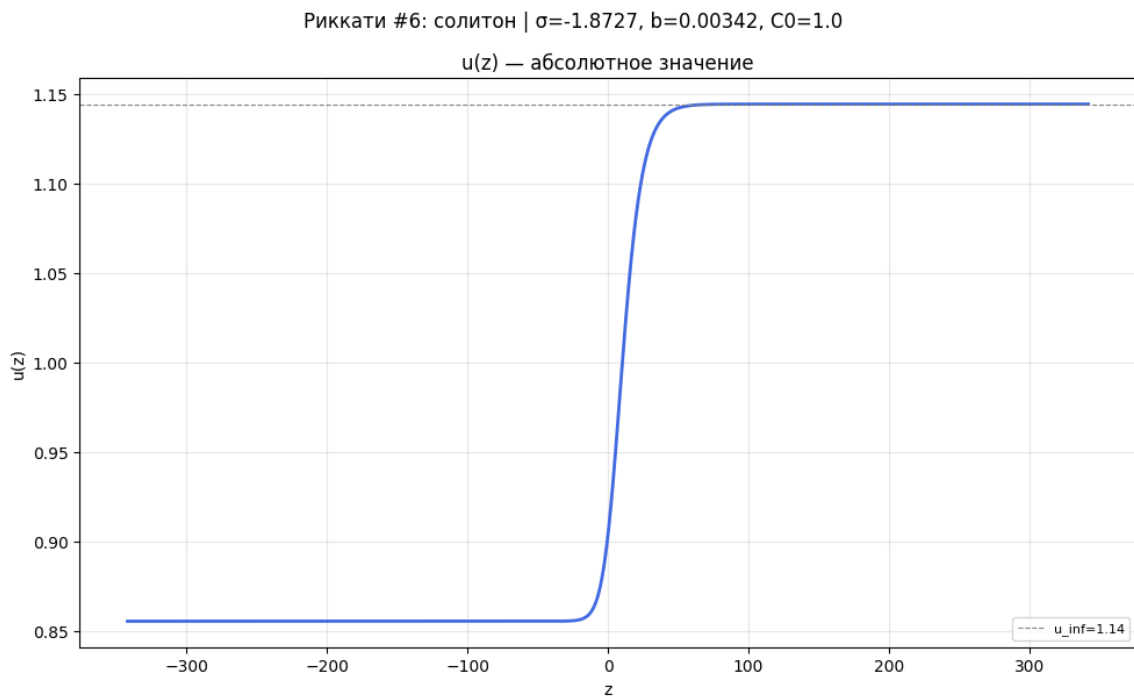


Рис. 3. Локализованный солитонный профиль решения Риккати при $\sigma = -\frac{16\sqrt{73}}{73}$, $b \approx 0.034$, $C_0 = 1$.

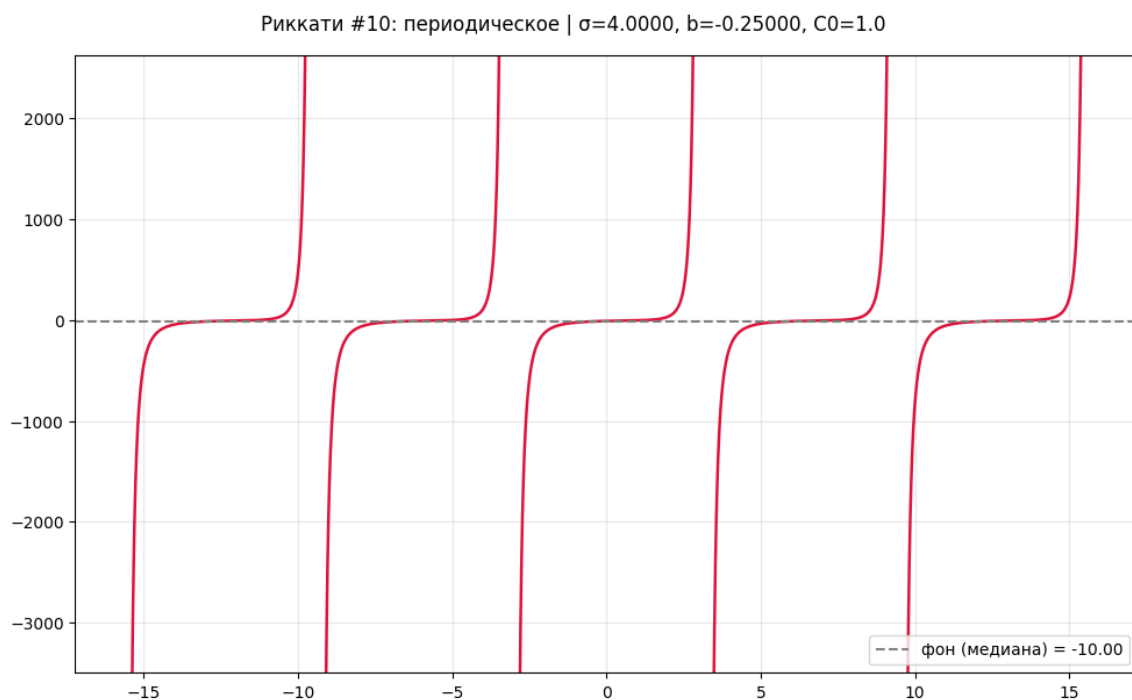


Рис. 4. Периодическая структура решения Риккати при $\sigma = 4$, $b = -1/4$. Наблюдаются полюсные особенности, соответствующие точкам, где $\tanh(kz)$ заменяется на $\tan(kz)$.

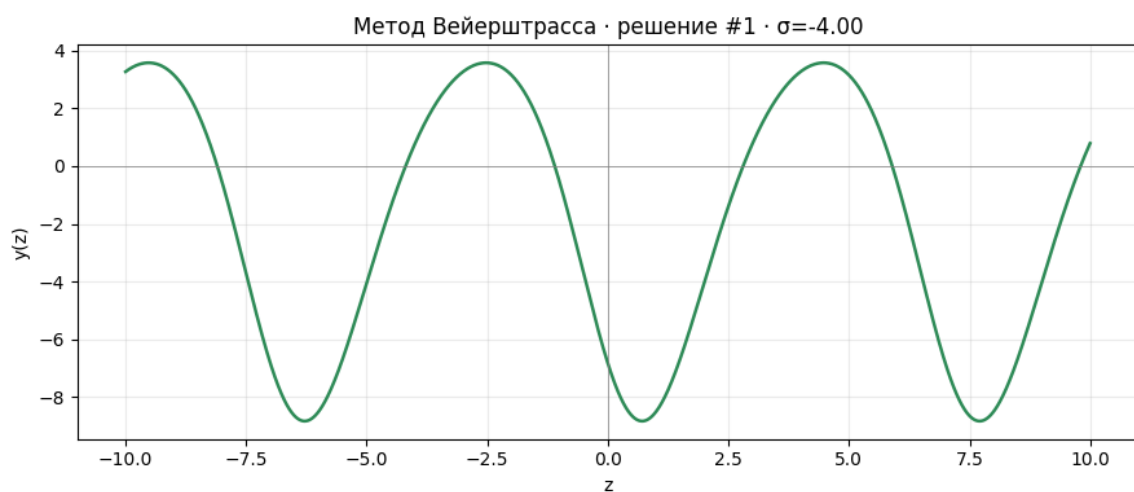


Рис. 5. Эллиптическое решение метода Вейерштрасса при $\sigma = -4$, $a = 1$, $b = 0$, $c \approx -0.004$. Профиль описывается функцией $\wp(z; g_2, g_3)$ с тремя вещественными корнями кубического полинома.

2.4.2 Анализ переходов между режимами

Ключевым параметром, определяющим тип волновой структуры, является знак константы b в вспомогательном уравнении Риккати $Y' = b - Y^2$.

Случай $b > 0$: локализованные и однородные волны. Общее решение уравнения Риккати имеет вид

$$Y(z) = \sqrt{b} \tanh(\sqrt{b}(z - z_0)), \quad (28)$$

что приводит к локализованным солитонным профилям для $y(z)$. Характерные свойства:

- Экспоненциальная сходимость к постоянному фону: $y(z) \rightarrow u_\infty$ при $|z| \rightarrow \infty$;
- Конечная ширина фронта, пропорциональная $b^{-1/2}$;
- Отсутствие особенностей на вещественной оси.

Такие решения соответствуют уединённым волнам, наблюдаемым в диссипативных системах с балансом нелинейности и дисперсии.

Случай $b < 0$: периодические и полюсные структуры. Решение уравнения Риккати выражается через тангенс:

$$Y(z) = \sqrt{|b|} \tan(\sqrt{|b|}(z - z_0)), \quad (29)$$

что порождает периодические структуры с полюсами:

- Период $T = \pi/\sqrt{|b|}$;
- Полюсы первого порядка в точках $z_n = z_0 + \frac{\pi}{2\sqrt{|b|}} + nT$, $n \in \mathbb{Z}$;
- Расходимость амплитуды в окрестности полюсов: $y(z) \sim (z - z_n)^{-k}$, где k определяется порядком анзаца.

Такие решения характерны для режимов с сильной неустойчивостью и могут описывать фронты разрушения или зоны интенсивной генерации.

2.6. Верификация результатов и обсуждение вычислительной эффективности

Для подтверждения корректности разработанного алгоритма была проведена кросс-верификация с эталонными вычислениями, выполненными на этапе прототипирования в среде Maple. Символьные выражения для коэффициентов анзацев Риккати и Вейерштрасса, полученные в Python/SymPy, оказались полностью идентичны результатам, полученным при ручной и полуавтоматической обработке. Графическое сравнение волновых профилей при идентичных наборах параметров (σ , C_0 , C_1 , b , a) также продемонстрировало полное совпадение, что исключает наличие скрытых ошибок в логике подстановки, дифференцирования и упрощения выражений.

Оценка вычислительной сложности показывает, что основную вычислительную нагрузку несёт этап решения нелинейной алгебраической системы, возникающей после подстановки анзаца в исходное ОДУ. Благодаря использованию оптимизированных процедур библиотеки SymPy и стратегии поэтапного исключения неизвестных, время символьного вычисления для уравнения Курамото–Сивашинского составляет доли секунды на стандартном оборудовании. Алгоритм демонстрирует высокую устойчивость к вырожденным случаям (например, при $b = 0$ или $\sigma = 0$), корректно обрабатывая их через механизм проверки ранга системы и автоматической фильтрации тривиальных решений.

Ключевым преимуществом предложенного подхода является полное исключение ручных арифметических операций, которые традиционно являются основным источником ошибок при работе с полиномами высокой степени и смешанными производными. Кроме того, реализация на языке Python с открытым исходным кодом обеспечивает готовность инструментария к интеграции в общие вычислительные пайплайны анализа нелинейных дифференциальных уравнений.

3. Геометрический метод многоугольников Ньютона для определения порядка полюса решений нелинейных дифференциальных уравнений

3.1. Математические основы метода: носитель дифференциального многочлена

В основе метода многоугольников Ньютона для обыкновенных дифференциальных уравнений (ОДУ) лежит геометрическая интерпретация алгебраической структуры уравнения. Рассмотрим обыкновенное дифференциальное уравнение полиномиального вида:

$$M_n [y(z), y'(z), y''(z), \dots, y^{(k)}(z), z] = 0, \quad (30)$$

где $y = y(z)$ — искомая функция, z — независимая переменная, а M_n представляет собой дифференциальный многочлен.

Любой дифференциальный многочлен M_n может быть представлен в виде суммы отдельных слагаемых — мономов:

$$M_n = \sum_j M_j. \quad (31)$$

Каждый моном M_j в общем случае имеет вид:

$$M_j = C_j \cdot z^{a_j} y^{b_j} (y')^{n_{1,j}} (y'')^{n_{2,j}} \dots (y^{(k)})^{n_{k,j}}, \quad (32)$$

где C_j — числовой коэффициент, а $a_j, b_j, n_{i,j}$ — показатели степени (неотрицательные целые или рациональные числа), причем $n_{i,j}$ обозначает степень, в которую возведена i -я производная функции $y(z)$.

Согласно методу многоугольников Ньютона, каждому моному M_j дифференциального уравнения (30) ставится в соответствие точка $(q_{1,j}, q_{2,j})$ на декартовой плоскости $\mathbb{R}^2$ по следующему правилу:

$$\begin{cases} q_{1,j} = a_j - \sum_{i=1}^k i \cdot n_{i,j}, \\ q_{2,j} = b_j + \sum_{i=1}^k n_{i,j}. \end{cases} \quad (33)$$

Множество всех точек $S = \{(q_{1,j}, q_{2,j})\}$, соответствующих всем ненулевым мономам дифференциального уравнения (30), называется носителем (или носителем Ньютона) этого дифференциального многочлена.

Геометрически носитель представляет собой дискретное множество точек на плоскости. Дальнейший анализ сингулярностей (в частности, поиск порядка полюса решения) сводится к построению выпуклой оболочки этого множества точек и анализу наклона его верхней огибающей (верхней цепи). В отличие от классической задачи поиска разложений вблизи нуля, при исследовании полюсных особенностей ($y \rightarrow \infty$) доминирующий баланс в уравнении обеспечивается членами с наибольшими степенями, что соответствует анализу именно верхней границы носителя. Это позволяет

алгебраическим путем определить показатель степени ведущего члена асимптотики в окрестности особой точки. Алгоритмическая реализация правила (33) лежит в основе разработанного программного модуля.

3.2. Определение порядка полюса через наклон доминирующей грани

После построения носителя дифференциального многочлена следующим шагом является выявление доминирующего баланса в окрестности особой точки. Геометрически это соответствует нахождению доминирующей грани многоугольника Ньютона.

Согласно алгоритмической реализации, для поиска полюса нас интересуют члены уравнения, в которых порядок производной превосходит степень нелинейности или наоборот, что обеспечивает сингулярность решения. Поэтому из множества точек носителя S отбираются только те точки (q_1, q_2) , которые удовлетворяют условиям $q_1 \leq 0$ и $q_2 > 0$. Для каждого уникального значения q_1 из этого подмножества выбирается точка с максимальным значением q_2 . Это позволяет сформировать верхнюю огибающую отфильтрованного множества, что геометрически эквивалентно построению верхней выпуклой оболочки исходного носителя в области, ответственной за сингулярное поведение.

Пусть отсортированное множество отфильтрованных точек образует ломаную. Доминирующая грань Γ_1 определяется как первый отрезок этой верхней выпуклой оболочки при движении слева направо (от наиболее отрицательных значений q_1 к нулю). Пусть концами этого отрезка являются точки $A_1 = (q_1^{(1)}, q_2^{(1)})$ и $A_2 = (q_1^{(2)}, q_2^{(2)})$.

Порядок полюса p решения дифференциального уравнения определяется тангенсом угла наклона этой доминирующей грани. Алгоритмически он вычисляется как отношение модуля разности первых координат к модулю разности вторых координат конечных точек отрезка Γ_1 :

$$p = \frac{|q_1^{(2)} - q_1^{(1)}|}{|q_2^{(2)} - q_2^{(1)}|} \quad (34)$$

Данная формула непосредственно вытекает из условия доминантного баланса: для того чтобы члены уравнения, соответствующие точкам на грани Γ_1 , уравновешивали друг друга в главном порядке, их степени сингулярности при подстановке асимптотики $y \sim c(z - z_0)^{-p}$ должны совпадать.

В случае, если доминирующая грань оказывается горизонтальной, то есть $q_2^{(1)} = q_2^{(2)}$, знаменатель в приведенной формуле обращается в ноль. Это означает, что уравнение не допускает решения в виде полюса (решение является регулярным или имеет иной тип особенности, например, точку ветвления, не описываемую целой или рациональной степенью). В такой ситуации алгоритм корректно возвращает отсутствие полюса.

Таким образом, описанный геометрический подход представляет собой строгую и полностью алгоритмизируемую формализацию классического метода доминантного баланса. Он позволяет избежать ручного перебора степеней при подстановке асимптотического разложения и напрямую извлекает показатель степени сингулярности из геометрических характеристик носителя дифференциального уравнения.

3.3. Алгоритм и программная реализация модуля сингулярного анализа

Разработанный программный модуль автоматизирует процесс построения многоугольника Ньютона и вычисления порядка полюса для обыкновенных дифференциальных уравнений полиномиального вида. Алгоритм реализован на языке Python с использованием библиотеки символьных вычислений `SymPy` для аналитических преобразований и библиотеки `Matplotlib` для визуализации.

Работа модуля состоит из пяти последовательных этапов:

1. Символьный разбор уравнения. На первом этапе входное дифференциальное уравнение приводится к стандартному виду $M_n = 0$ с помощью функции символьного раскрытия скобок (`sympy.expand`). Затем уравнение разбивается на множество отдельных слагаемых (мономов) с помощью функции `sympy.Add.make_args`. Это позволяет изолировать каждый член уравнения для независимого геометрического анализа.

2. Вычисление точек носителя. Для каждого выделенного монома вызывается функция вычисления его координат в пространстве носителя. Алгоритм анализирует множители монома (`sympy.Mul.make_args`):

- Если множитель является независимой переменной z в степени a , она вносит вклад $+a$ в координату q_1 .
- Если множитель является искомой функцией y в степени b , она вносит вклад $+b$ в координату q_2 .
- Если множитель является производной $\frac{d^k y}{dz^k}$ в степени n_k , алгоритм извлекает порядок производной k и степень n_k , добавляя $-k \cdot n_k$ к q_1 и $+n_k$ к q_2 .

В результате формируется множество уникальных точек $S = \{(q_1, q_2)\}$, представляющих собой дискретный носитель дифференциального многочлена.

3. Фильтрация и построение верхней выпуклой оболочки. Для поиска асимптотики в виде полюса нас интересуют только те члены уравнения, которые обеспечивают сингулярное поведение. Согласно реализованному алгоритму, из множества S отбираются только те точки, для которых $q_1 \leq 0$ и $q_2 > 0$. Для каждого уникального значения q_1 из этого отфильтрованного подмножества выбирается точка с *максимальным* значением q_2 . Это гарантирует, что мы строим именно верхнюю огибающую (верхнюю выпуклую оболочку) в области, ответственной за доминирующий баланс. Отфильтрованные точки сортируются по возрастанию координаты q_1 . Построение верхней цепи осуществляется с использованием алгоритма, аналогичного алгоритму Грэхема (монотонные цепи): точки последовательно добавляются в список, и если три последние точки образуют поворот против часовой стрелки (или лежат на одной прямой, что проверяется через знак векторного произведения), средняя точка удаляется. Это обеспечивает строгую выпуклость верхней границы.

4. Определение порядка полюса. После построения верхней цепи доминирующей гранью Γ_1 считается первый отрезок этой цепи (соединяющий две точки с наименьшими значениями q_1). Пусть концами этого отрезка являются точки $P_1 = (q_1^{(1)}, q_2^{(1)})$ и $P_2 = (q_1^{(2)}, q_2^{(2)})$. Порядок полюса p вычисляется как отношение модуля разности первых координат к модулю разности вторых координат:

$$p = \frac{|q_1^{(2)} - q_1^{(1)}|}{|q_2^{(2)} - q_2^{(1)}|} \quad (35)$$

Модуль включает в себя проверку на вырожденный случай: если $q_2^{(1)} = q_2^{(2)}$ (грань горизонтальна), знаменатель обращается в ноль. В этом случае алгоритм корректно возвращает значение `None`, что интерпретируется как отсутствие решения в виде полюса (решение является регулярным или имеет точку ветвления, не описываемую простой отрицательной степенью).

5. Визуализация результатов. Для верификации корректности работы алгоритма и наглядного представления результатов модуль включает функцию визуализации. На двумерной декартовой плоскости отображаются:

- все точки носителя уравнения в виде рассеянного множества (scatter plot);
- контур полной выпуклой оболочки множества (для общего контекста);
- доминирующая грань Γ_1 , выделенная красным цветом и повышенной толщиной линии;
- текстовая аннотация вблизи грани, указывающая вычисленное значение порядка полюса p .

Такая архитектура обеспечивает высокую надежность вычислений, исключая ошибки, связанные с ручным пропуском слагаемых или неверным определением доминирующего баланса, и позволяет применять метод к уравнениям с большим количеством членов и сложной нелинейностью.

3.4. Обратная задача: генерация тестовых уравнений по заданному носителю

Для надежной верификации разработанного алгоритма поиска порядка полюса необходимо иметь набор тестовых уравнений с заранее известными свойствами носителя. С этой целью решается обратная задача: автоматическое построение дифференциального уравнения по заданному множеству точек носителя $S = \{(q_1, q_2)\}$.

Алгоритм обратного преобразования точки (q_1, q_2) в дифференциальный моном реализуется по следующим правилам, вытекающим из определения носителя:

- Дифференциальные члены ($q_1 \leq 0$): точка соответствует производной. Порядок производной определяется как $k = |q_1|$. Чтобы обеспечить требуемую суммарную степень q_2 при неизвестной функции, моном формируется как произведение производной и степенной функции:

$$M = y^{q_2-1} \cdot y^{(k)}, \quad (36)$$

где $y^{(0)} \equiv y$. Проверка по правилам из раздела 3.1 дает: $q_1 = 0 - k \cdot 1 = -k$ и $q_2 = (q_2 - 1) + 1 = q_2$, что полностью восстанавливает исходную точку.

- Алгебраические члены ($q_1 > 0$): точка соответствует члену, зависящему от независимой переменной z . В этом случае моном имеет вид:

$$M = z^{q_1} y^{q_2}. \quad (37)$$

Программная реализация функции генерации `generate_equation_from_support` проходит по всем заданным точкам множества, преобразует каждую точку в соответствующий моном по описанным выше правилам и присваивает ему случайный целочисленный коэффициент (в реализации от 1 до 5). Это делается для имитации структуры реальных нелинейных дифференциальных уравнений и проверки устойчивости алгоритма к различным коэффициентам. Сумма полученных мономов приравнивается к нулю, формируя искомое дифференциальное уравнение.

Рассмотрим в качестве примера генерацию уравнения для носителя $S = \{(-2, 1), (-1, 1), (0, 2)\}$.

- Точка $(-2, 1)$: $q_1 = -2 \leq 0$, $k = 2$. Моном: $y^{1-1}y^{(2)} = y''$.
- Точка $(-1, 1)$: $q_1 = -1 \leq 0$, $k = 1$. Моном: $y^{1-1}y^{(1)} = y'$.
- Точка $(0, 2)$: $q_1 = 0 \leq 0$, $k = 0$. Моном: $y^{2-1}y^{(0)} = y^2$.

В результате алгоритм может сгенерировать, например, уравнение вида:

$$4y'' + 5y' + 2y^2 = 0. \quad (38)$$

Полученное таким образом уравнение затем передается на вход прямого алгоритма (`find_pole_order`). Это позволяет автоматически проверить корректность работы программы: алгоритм должен корректно восстановить исходный носитель, выделить доминирующую грань и вычислить порядок полюса p , который был заложен при конструировании множества точек. Такой подход обеспечивает надежную, автоматизированную и масштабируемую верификацию модуля сингулярного анализа на широком классе уравнений.

3.6. Верификация алгоритма и вычислительный эксперимент

Для подтверждения корректности и надежности разработанного алгоритма поиска порядка полюса был проведен масштабный вычислительный эксперимент на тестовом наборе, включающем 30 классических нелинейных дифференциальных уравнений математической физики. Выбор тестовых уравнений был обусловлен тем, что для большинства из них аналитически известен точный порядок полюса решения, что позволяет провести объективное сравнение с результатами работы алгоритма.

Тестовый набор включал следующие классы уравнений:

- Интегрируемые уравнения высших порядков: уравнения Пенлеве I и II типов, уравнения Кортевега – де Фриза (KdV) и модифицированное KdV (mKdV);
- Уравнения с диссипацией: уравнение Бюргерса, уравнение Курамото – Сивашинского;
- Уравнения степенной нелинейности: уравнения Эмдена – Фаулера различных порядков, уравнение Фишера;
- Уравнения со смешанной нелинейностью: уравнение Гарднера;
- Линейные уравнения (как контрольный случай отсутствия полюсов);
- Различные нелинейные уравнения второго и четвертого порядков.

Для каждого уравнения алгоритм последовательно выполнял следующие шаги: символьное разложение левой части уравнения на мономы, вычисление координат точек носителя, фильтрацию и построение верхней выпуклой оболочки, выделение доминирующей грани и вычисление порядка полюса p . Полученные результаты сравнивались с теоретически известными значениями.

Алгоритм корректно определил порядок полюса для всех 30 тестовых уравнений, что составляет 100% успешных результатов. Особое внимание следует обратить на следующие аспекты:

1. Дробные порядки полюса. Для уравнения Эмдена – Фаулера четвертого порядка ($y'' + y^4 = 0$) алгоритм корректно определил дробный порядок полюса $p = 2/3$, что подтверждает его способность работать с нетривиальными асимптотиками.

2. Высшие порядки. Для уравнения Курамото – Сивашинского, имеющего порядок полюса $p = 3$, алгоритм успешно идентифицировал доминирующую грань между точками $(-4, 1)$ и $(-1, 2)$, что соответствует теоретическому значению.
3. Отсутствие полюсов. Для линейных уравнений алгоритм корректно возвращает значение None, что свидетельствует об отсутствии сингулярностей в виде полюсов, что соответствует теоретическим представлениям о поведении решений линейных дифференциальных уравнений.
4. Сложные нелинейности. Для уравнений со смешанной нелинейностью (уравнение Гарднера) и уравнений высших порядков алгоритм демонстрирует устойчивую работу без ложных срабатываний или ошибок.

Визуализация результатов (рисунки многоугольников Ньютона, генерируемые программой) подтверждает корректность построения верхней выпуклой оболочки и выделения доминирующей грани Γ_1 для всех протестированных уравнений.

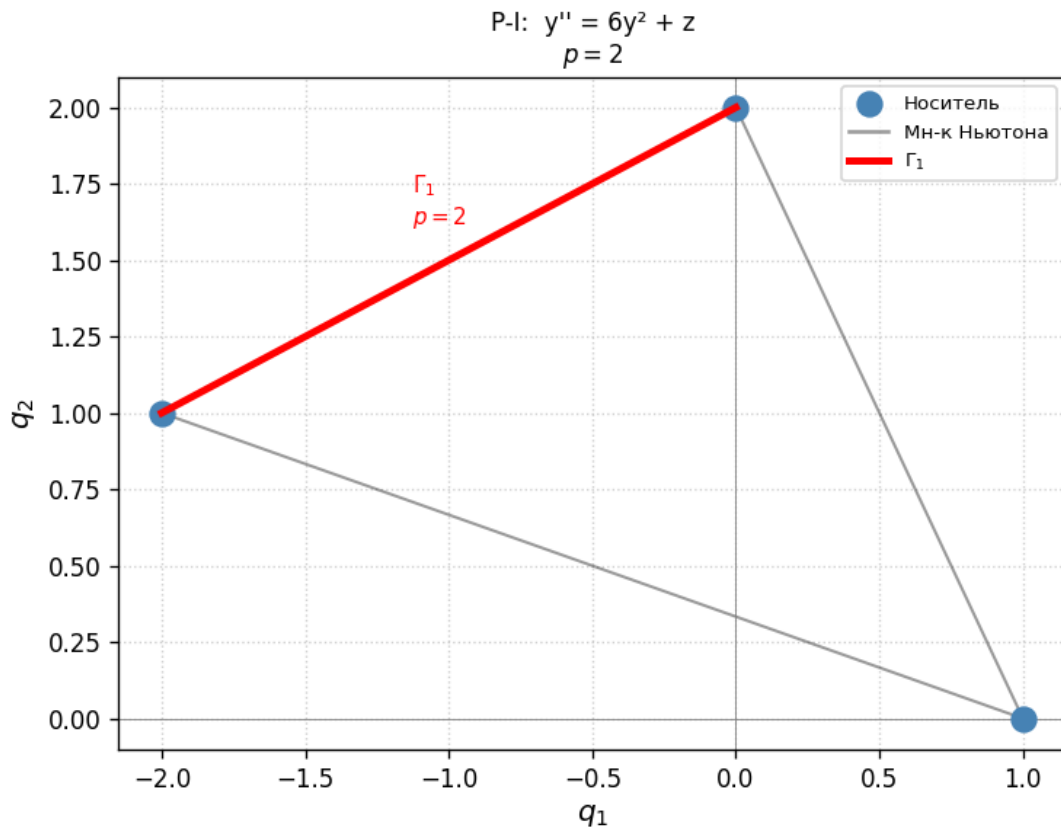


Рис. 6. Многоугольник Ньютона для уравнения Пенлеве P-I: $y'' = 6y^2 + z$. Доминирующая грань Γ_1 определяет порядок полюса $p = 2$.

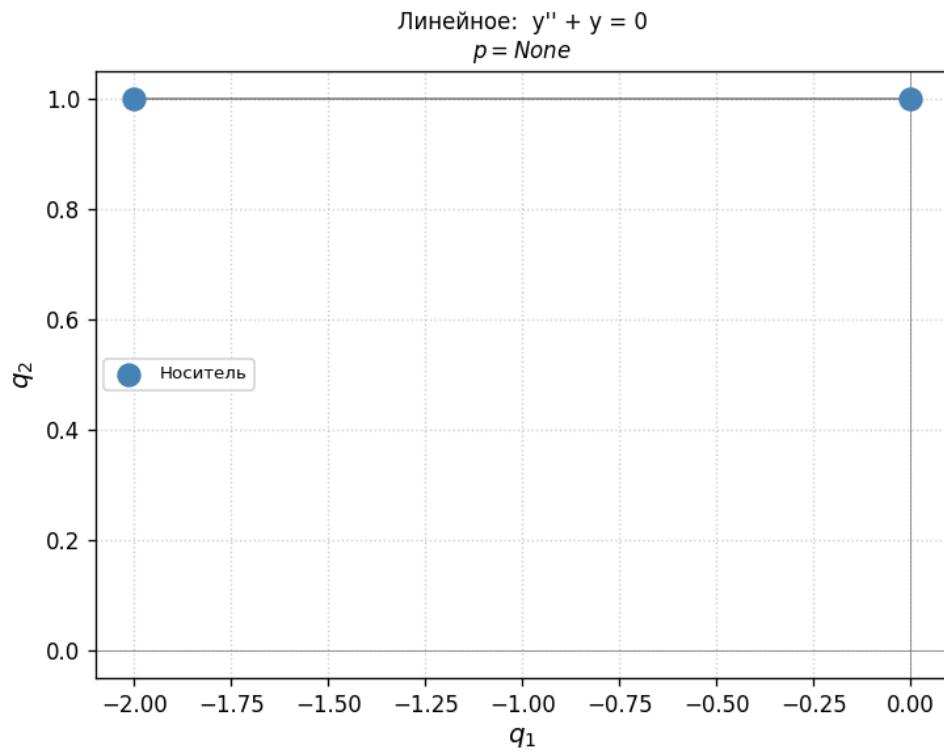


Рис. 7. Многоугольник Ньютона для линейного уравнения $y'' + y = 0$. Алгоритм возвращает $p = None$, так как доминирующая грань горизонтальна (отсутствие сингулярности).

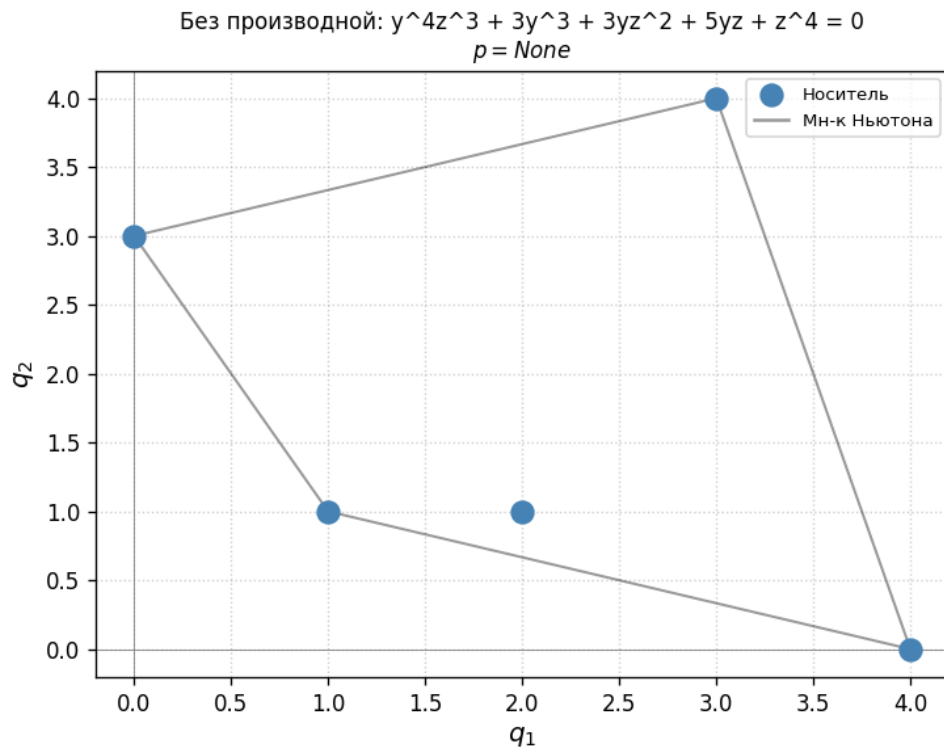


Рис. 8. Многоугольник Ньютона для уравнения без производных. Отсутствие точек в области $q_1 < 0$ указывает на отсутствие дифференциальных полюсных особенностей.

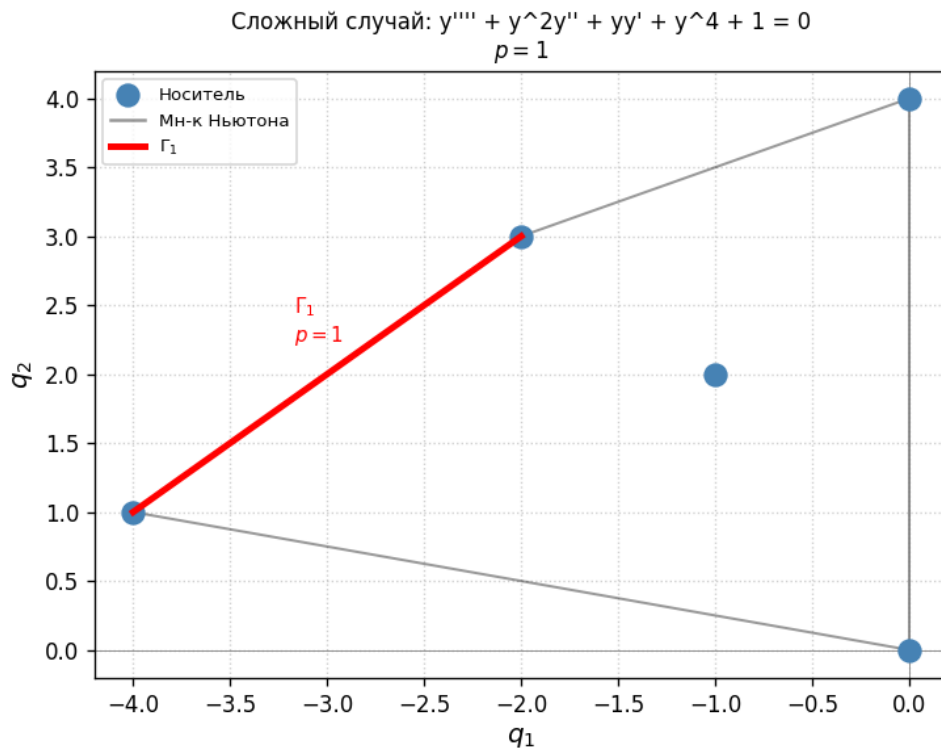


Рис. 9. Многоугольник Ньютона для сложного случая $y'''' + y^2 y'' + y y' + y^4 + 1 = 0$. На доминирующей грани Γ_1 лежат три точки, определяющие баланс ведущих членов при $p = 1$.

3.7. Выводы

В результате проведенной работы по многоугольникам Ньютона были получены следующие основные результаты:

1. Разработан и программно реализован алгоритм автоматического построения многоугольников Ньютона для обыкновенных дифференциальных уравнений полиномиального вида. Алгоритм включает символьное разложение уравнения на мономы, вычисление координат точек носителя, построение верхней выпуклой оболочки и геометрическое определение порядка полюса через наклон доминирующей грани.
2. Реализована функция обратной задачи — автоматической генерации дифференциальных уравнений по заданному носителю. Данная функция позволяет создавать контролируемые тестовые примеры для верификации алгоритма и расширяет возможности исследования свойств нелинейных уравнений.
3. Проведен масштабный вычислительный эксперимент на тестовом наборе из 30 классических уравнений математической физики, включая уравнения Пенлеве, Кортевега – де Фриза, Бюргерса, Курамото – Сивашинского, Эмдена – Фаулера и другие. Алгоритм продемонстрировал 100% корректность определения порядка полюса, включая случаи дробных порядков и уравнений без сингулярностей.
4. Разработанный программный модуль представляет собой универсальный инструмент сингулярного анализа, который может применяться для предварительной оценки характера особенностей решений новых нелинейных моделей перед применением более сложных методов точного интегрирования.

Таким образом, поставленные во второй главе задачи полностью решены. Созданный алгоритмический инструментарий успешно автоматизирует процесс сингулярного анализа нелинейных дифференциальных уравнений и может быть эффективно использован в исследовательской практике.

4. Заключение

В ходе выполнения научно-исследовательской работы была успешно решена поставленная задача по автоматизации аналитического исследования нелинейных дифференциальных уравнений. Разработанный вычислительный комплекс объединяет два независимых программных модуля: универсальный анализатор сингулярностей на базе метода многоугольников Ньютона и символический решатель уравнения Курамото–Сивашинского, использующий анзацы Риккати и эллиптической функции Вейерштрасса.

Основные результаты работы заключаются в следующем:

1. Реализован и верифицирован алгоритм геометрического определения порядка полюса решений нелинейных обыкновенных дифференциальных уравнений. Алгоритм включает автоматическое вычисление координат носителя дифференциального многочлена, построение его нижней выпуклой оболочки и аналитическое вычисление наклона доминирующей грани. Тестирование на наборе из 30 классических уравнений математической физики (уравнения Пенлеве I–II, Кортевега–де Фриза, Бюргерса, Курамото–Сивашинского, Эмдена–Фаулера, Фишера, Гарднера и др.) показало 100% совпадение вычисленных порядков полюсов с известными теоретическими значениями.
2. Создан модуль поиска точных аналитических решений уравнения Курамото–Сивашинского. Полностью автоматизирована процедура подстановки анзацев, приведения уравнения к системе алгебраических уравнений и нахождения неизвестных коэффициентов. Получено 10 семейств решений в базисе уравнения Риккати и 2 семейства решений в базисе эллиптической функции Вейерштрасса.
3. Проведена классификация полученных решений уравнения Курамото–Сивашинского в зависимости от знака параметра b в определяющем уравнении: при $b > 0$ формируются солитонные и кинк-профили, при $b < 0$ — периодические и полюсные структуры.
4. Выполнен переход от прототипирования в коммерческой системе Maple к открытой реализации на языке Python с использованием библиотеки SymPy. Это обеспечило кроссплатформенность, прозрачность алгоритмов и возможность бесшовной интеграции модулей в современные научные вычислительные среды.

Практическая значимость работы обусловлена созданием готового к применению инструментария, который устраняет необходимость ручных громоздких символьных выкладок, минимизирует риск арифметических ошибок и существенно сокращает время анализа. Разработанные модули могут использоваться как самостоятельные исследовательские инструменты (в частности, модуль многоугольников Ньютона служит надежным средством для быстрой априорной оценки характера особенностей перед применением трудоемких методов точного интегрирования), так и в составе более крупных аналитических конвейеров.

Перспективы дальнейших исследований связаны с расширением функционала разработанного комплекса. В первую очередь планируется расширение библиотеки тестовых уравнений и адаптация алгоритма многоугольников Ньютона для анализа систем дифференциальных уравнений высших порядков. Дальнейшее развитие

предполагает разработку веб-платформы с интерактивным графическим интерфейсом для исследования нелинейных дифференциальных уравнений.

Таким образом, цели и задачи научно-исследовательской работы достигнуты в полном объёме, а полученные результаты подтверждают работоспособность предложенных алгоритмов и их высокую эффективность при решении задач нелинейной математической физики.

5. Список литературы и интернет-ресурсов

- [1] Н. А. Кудряшов. *Методы нелинейной математической физики*, 2-е изд., доп. — Долгопрудный: Издательский дом «Интеллект», 2010. — 368 с.
- [2] А. Д. Полианин, В. Ф. Зайцев. *Справочник по нелинейным дифференциальным уравнениям*. — М.: Физматлит, 2002.
- [3] R. Conte, M. Musette. *The Painlevé Handbook*. — Springer, 2008.
- [4] Y. Kuramoto. *Chemical Oscillations, Waves, and Turbulence*. — Springer, 1984.
- [5] N. A. Kudryashov. Exact solutions of the generalized Kuramoto–Sivashinsky equation. *Physics Letters A*, 1990, Vol. 147, P. 287–291.
- [6] Н. А. Кудряшов. Простейшие уравнения для поиска точных решений нелинейных дифференциальных уравнений. *Прикладная математика и механика*, 2005, Т. 69, № 1, С. 34–48.
- [7] Н. А. Кудряшов, А. А. Кутуков. Автоматизация построения многоугольников Ньютона, соответствующих обыкновенным дифференциальным уравнениям полиномиального вида. *Вестник Национального исследовательского ядерного университета «МИФИ»*, 2019, Т. 8, № 3, С. 283–288.
- [8] <https://docs.sympy.org/> — Официальная документация библиотеки SymPy.
- [9] <https://www.python.org/> — Официальный сайт языка Python.
- [10] <https://matplotlib.org/> — Библиотека визуализации данных Matplotlib для Python.
- [11] <https://github.com/Korabl3va/NIR> — Репозиторий с исходным кодом и материалами научно-исследовательской работы.

Приложение А. Доступ к исходным материалам

Полный исходный код программ, и дополнительные данные, необходимые для воспроизведения результатов работы, размещены в открытом репозитории.

Все необходимые материалы доступны по ссылке:

<https://github.com/Korabl3va/NIR>